Which Contributions Predict whether Developers are Accepted Into Github Teams

Nikhil Anand

April 22, 2026

The rapid growth of open-source software (OSS) has transformed the way software is developed, enabling globally distributed collaboration and widespread access to code and innovation. Platforms like GitHub have become central to this ecosystem, hosting millions of projects and developers who contribute in diverse ways—from reporting issues and engaging in discussions to submitting code changes. However, despite the openness of OSS communities, not all contributors transition into core team members. Many projects struggle to balance inclusivity with maintaining high-quality contributions, making it difficult to determine which types of developer interactions actually lead to team acceptance. This challenge highlights a critical gap in understanding how contributors evolve from peripheral participants to trusted collaborators within OSS teams.

To address this gap, the paper investigates the patterns of developer activity on GitHub that distinguish those who successfully join project teams from those who remain outsiders. By analyzing large-scale interaction data—including pull requests, comments, and other forms of engagement—the study aims to identify which behaviors are most predictive of joining outcomes. Rather than focusing solely on code contributions, the work considers a broader spectrum of social and technical interactions, emphasizing the role of communication and visibility in collaborative environments. Ultimately, the study seeks to provide insights that can inform OSS platform design, community practices, and developer education, helping both contributors and teams better navigate the process of onboarding and collaboration.

1 Motivation

The motivation for this paper arises from the increasing importance of open-source software (OSS) in modern software development and the growing reliance of society on OSS-based systems. The proliferation of the Internet and OSS philosophies has enabled developers across the globe to collaboratively build and maintain large-scale systems, such as the Linux kernel and Mozilla Firefox [1, 2, 3]. While this openness lowers barriers to entry and encourages widespread participation, it also introduces challenges in sustaining these projects. Many OSS teams struggle to attract and retain sufficient contributors, which can lead to serious consequences for software quality and security. A notable example is the Heartbleed vulnerability in OpenSSL, where a small, overstretched team failed to detect a critical bug affecting a large portion of the web [4]. This illustrates that simply having open access to contributions is not enough; projects must also ensure that contributors are capable and trustworthy, creating a tension between inclusivity and quality control [5].

This tension motivates a deeper investigation into how developers transition from being external contributors to becoming trusted members of OSS teams. Although collaborative platforms like GitHub provide mechanisms for interaction—such as pull requests, issue discussions, and social signals—they also create a complex ecosystem where it is difficult to identify which behaviors actually lead to team acceptance. With millions of users and projects, GitHub serves as a rich environment where diverse forms of developer activity can be observed at scale. However, despite the abundance of data, there remains a lack of clarity on what distinguishes successful contributors from those who remain on the periphery. This gap in understanding limits both developers, who seek to integrate into projects, and teams, who aim to identify and recruit valuable contributors effectively.

Furthermore, prior research has explored OSS communities through structural and social lenses, such as core-periphery models and joining scripts, but often lacks quantitative analysis of the specific contribution behaviors that drive team acceptance [1]. Existing studies frequently emphasize social connections or initial interactions rather than the sustained patterns of activity that characterize successful contributors. As a result, there is a need for a more comprehensive and data-driven understanding of how different types of contributions—ranging from passive attention to active code submissions and discussions—impact the likelihood of joining a team. This paper is motivated by the opportunity to leverage large-scale GitHub data to systematically analyze these behaviors and uncover actionable insights.

Ultimately, the motivation extends beyond academic curiosity to practical implications for the OSS ecosystem. By identifying which forms of interaction most strongly influence team acceptance, the study aims to inform the design of collaborative platforms, guide educators in preparing developers for OSS participation, and help contributors adopt more effective engagement strategies. Understanding these dynamics is essential for improving the sustainability and resilience of OSS projects, which increasingly underpin critical infrastructure and software systems worldwide.

1.1 Related Work

1.1.1 The Structures of OSS Teams

Research on OSS communities commonly models their structure using a core periphery or "onion" model, where developers are organized into layers based on their level of involvement and responsibility. At the outermost layers are passive users and occasional contributors, while the innermost layers consist of core developers and project leaders who actively maintain the system [1]. Studies have shown that this structure is critical for the health of OSS projects, as it balances participation and responsibility across different roles [6, 7]. Importantly, contributions do not just modify the codebase but also influence the social structure of the community by redefining contributors' roles and positions within it [1]. However, some researchers argue that this model does not fully capture the dynamic nature of OSS communities, where developers frequently move between layers rather than remaining fixed in one position [8].

Further work highlights that membership in OSS teams is not uniformly defined and varies across projects. Some perspectives consider anyone who contributes or provides meaningful input as part of the team, while others emphasize formal roles such as commit access or "committer" status as indicators of true membership [9, 10]. Additionally, studies suggest that core developers are typically those who contribute significant and frequent code changes, although formal designations do not always align perfectly with perceived roles [11, 12, 13]. These variations underscore the complexity of defining team boundaries in OSS and motivate the need for standardized, observable definitions of membership—such as role assignments on platforms like GitHub—to enable large-scale empirical analysis.

1.1.2 The Theories of OSS Joining

A key line of research investigates how developers transition from peripheral contributors to core team members. One influential concept is the "joining script," which describes a general sequence of activities that newcomers follow to gain acceptance into a project [14]. This script is not rigid but reflects common behavioral patterns, such as starting with bug reporting or minor contributions and gradually increasing involvement. Empirical studies support this idea, showing that successful joiners often engage in specific activities—like fixing bugs or contributing to discussions—while non-joiners may exhibit more limited or less targeted participation [15]. These findings suggest that joining is shaped by both the type and progression of contributions rather than isolated actions.

Other research expands this perspective by examining factors beyond direct contributions, including social connections, prior experience, and visibility within the community. For example, having existing ties to core developers or prior contributions to other OSS projects significantly increases the likelihood of joining [15, 16]. Additionally, early engagement behaviors, such as watching projects, can indicate future involvement [17]. However, not all contributors follow the same trajectory; differences exist between volunteer contributors and those employed by organizations to work on OSS, leading to distinct joining patterns [18]. Despite these insights, much of the prior work focuses on qualitative analysis or specific case

studies, leaving a gap in quantitatively understanding how different types of contributions—especially their frequency and consistency—affect joining outcomes.

1.1.3 Using GitHub for OSS Research

GitHub has become a central platform for studying OSS due to its transparency and the richness of interaction data it provides. Researchers have emphasized the concept of "social translucence," where visible traces of user activity—such as commits, pull requests, and comments—allow others to infer a developer's skills, interests, and level of engagement [19]. These activity traces play a critical role in shaping how contributors are perceived and evaluated within the community. Additional studies show that users form impressions based on both technical contributions and social signals, which influence decisions such as accepting code changes or inviting contributors to collaborate more closely [20].

Beyond individual evaluation, GitHub also enables researchers to study how social and technical dynamics influence broader community behavior. For instance, prior work has examined how norms, such as testing practices, spread within projects and how social interactions facilitate or hinder this process [21]. However, much of this research relies on qualitative methods, such as interviews or case studies, to understand how GitHub's interface shapes collaboration. In contrast, there is a need for large-scale quantitative analyses that connect these observable activities to concrete outcomes, such as team formation and developer onboarding. This gap motivates the current study's use of GitHub as a unified platform to systematically analyze contribution patterns across thousands of projects.

1.2 Research Questions

As the influence of OSS projects grows and more consumers depend on their outcomes, the necessity of keeping these projects well-maintained becomes all the more critical. But we do not know of,nor is there likely to be an exact and universal process to join development teams; the requirements for joining a software team varies between project types and team culture. The authors pose three research questions

RQ1: What types and qualities of project interactions distinguish between developers who join an OSS team and developers who remain outside?

RQ2: What activities or qualities of a developer independent from a specific team influence whether the developer joins that team?

RQ3: What activities or qualities of a team influence whether the team accepts more people in?

2 Methodology

2.1 Data Selection and Collection

The study selects GitHub as the primary data source due to its **popularity, scale, and transparency**, making it a representative platform for OSS activity. Data is collected from two snapshots of the GHTorrent dataset (August 2015 and January 2017), enabling the researchers to **track developer transitions over time**. Since certain information—such as team membership—is not fully available in GHTorrent, the authors supplement the dataset by **manually scraping GitHub webpages**. This combination of large-scale archival data and targeted manual collection allows the study to capture both **interaction-level activity and membership status** necessary for analyzing developer onboarding behavior.

2.2 Defining Membership on GitHub

The paper defines membership using the concept of the **core-periphery (onion) model**, where developers move from peripheral contributors to central team members. To operationalize this, the authors define "insiders" as developers with **write access** (e.g., Collaborators or Members) and "outsiders" as those without such privileges. This distinction is crucial because write access represents both **increased responsibility and trust from the team**, making it a meaningful indicator of successful onboarding.

To identify transitions, the study compares developer roles across two time snapshots and focuses on those who were outsiders in 2015 but became insiders by 2017. However, GitHub changed its role definitions over time, requiring the authors to **manually map roles across versions** using archived data. Additionally, since some membership information may be hidden due to privacy settings, the authors infer membership by checking for **exclusive actions such as merging pull requests or committing directly**, ensuring a more accurate classification.

Ultimately, the study defines "joiners" as developers who **gained insider status between the two snapshots**, while "non-joiners" remained outsiders. This approach allows the researchers to **systematically track onboarding behavior** across thousands of developer–team relationships, forming the basis for comparative analysis between successful and unsuccessful contributors.

2.3 GitHub Contributions

To analyze developer behavior, the study categorizes contributions into three perspectives: **interactions between a developer and a team (RQ1)**, **developer activity independent of the team (RQ2)**, and **team-level activity (RQ3)**. For each case, metrics are collected based on activity prior to the first snapshot, ensuring that the analysis focuses on **predictive behavior rather than post-joining activity**. This structured separation enables the study to disentangle the effects of individual behavior, relationship dynamics, and team characteristics.

At the interaction level, the study captures a wide range of activities available to outsiders, including **watching projects, forking repositories, opening issues, commenting in discussions, and submitting pull requests**. These actions are treated as measurable indicators of engagement, with particular attention to the distinction between **code contributions and social interactions**. The authors further refine discussion metrics by separating different types of comments, such as those on one's own pull requests versus others', to account for **different social dynamics in collaboration**.

At the developer and team levels, additional metrics are introduced to capture broader influence and context. For developers, this includes **commits to personal projects, number of projects created, followers, and social reach**. For teams, metrics such as **number of projects, contributors, commits, and community attention (watchers/forkers)** are used to characterize their scale and activity. Together, these metrics provide a comprehensive view of both **technical contributions and social signals** that may influence the likelihood of joining a team.

2.4 Data Cleaning

To ensure meaningful analysis, the dataset is filtered to include only **valid developer–team relationships where the developer was initially an outsider**. Relationships involving personal repositories or pre-existing team members are excluded to maintain focus on onboarding behavior. Additionally, the authors remove relationships with no recent activity, as these may reflect **stale interactions or unobserved external communication**, which could distort metrics like tenure.

Further cleaning involves identifying and removing outliers, particularly bot accounts, which were detected through naming patterns and abnormal activity levels. This step ensures that the dataset reflects **genuine human interactions rather than automated behavior**. By applying these filters, the study refines the dataset to include only **active, relevant, and reliable observations**, improving the validity of subsequent analysis.

2.5 Analysis

The study employs **logistic regression** to model the probability of a developer joining a team, as the outcome variable is binary (join vs. not join). To handle skewed distributions and reduce the influence of extreme values, all explanatory variables are **log-scaled**. This allows the model to better capture proportional effects of contributions across different activity levels.

To avoid bias from repeated observations of the same developers or teams, the authors design two

complementary analyses using **paired sampling**[1]. One analysis compares developers within the same team (team perspective), while the other compares teams for the same developer (developer perspective). This dual approach helps **isolate the effects of individual behavior versus team characteristics** and provides a more robust understanding of the factors influencing OSS team joining.

3 Findings and Discussions

3.1 How Do Developer-To-Team Interactions Influence Joining?

The study finds that among all interaction types, **pull request submissions have the strongest influence on whether a developer joins a team**. This highlights that **active code contribution is a critical signal of capability and commitment**. However, success is not driven by code contributions alone—**engagement in discussions, particularly commenting on others' pull requests, also significantly increases joining likelihood**. This suggests that teams value developers who participate in the collaborative process, not just those who contribute code in isolation. Additionally, broader discussion activities, such as issue comments, further reinforce the importance of **communication and social interaction** in OSS communities.

The findings also align with the onion model of OSS participation, where deeper involvement corresponds to higher likelihood of joining. Activities that require greater investment—like contributing code—are fewer but more impactful, whereas lighter activities like watching or forking are less influential. The study further shows that **tenure (time spent interacting with the project)** plays a meaningful role, indicating that **sustained engagement over time helps build trust and familiarity**. Importantly, the results suggest that joining is not about excelling in a single activity but rather **maintaining consistent participation across multiple forms of interaction**.

Overall, the key takeaway is that **more interaction is generally beneficial, but certain interactions matter more than others**. Developers who combine **frequent pull requests, active participation in discussions, and long-term engagement** are significantly more likely to transition into team members. This emphasizes that OSS collaboration is inherently social, and developers must demonstrate both technical ability and collaborative behavior to be accepted.

3.2 How Does Independent User Activity Influence Joining?

When examining developer activity independent of any specific team, the study finds that only a few factors significantly influence joining. Notably, **the number of commits to personal projects** emerges as a strong indicator, suggesting that **demonstrated coding experience and self-driven work enhance credibility**. This reflects that teams evaluate not only direct contributions but also a developer's broader portfolio and ability to independently build and maintain software.

Another important factor is **social visibility, particularly the number of followers a developer has**. A higher follower count acts as a signal of reputation, implying that **other developers already recognize and trust this individual's work**. This creates a form of "pre-validation," making teams more likely to accept such developers. However, this also introduces an imbalance, as developers with fewer followers may need to **put in more effort to prove their value**, highlighting the role of social dynamics in OSS participation.

These findings suggest that joining is influenced not just by direct interactions with a team but also by **a developer's overall presence and reputation on the platform**. Both **technical productivity (personal commits)** and **social recognition (followers)** contribute to onboarding success. This underscores the importance of building a visible and active profile in OSS ecosystems, as teams consider broader signals beyond immediate contributions.

[1]Paired sampling is a technique where observations are grouped into matched pairs (e.g., same developer or same team) to control for confounding factors and enable more reliable comparison between outcomes.

3.3 How Does Independent Team Activity Influence Joining?

From the perspective of team characteristics, the study reveals that certain team-level factors influence how likely developers are to join. Teams with **more projects, higher numbers of pull requests, and a larger contributor base** are generally more receptive to new members. These factors indicate **active and growing projects**, where additional contributors are needed to sustain development. In such environments, developers are more likely to find opportunities to contribute and be accepted.

However, the study also identifies an interesting contrast: teams with **high visibility, measured through watchers and forkers, or large numbers of commits** tend to be less likely to accept new members. This suggests that **popular or mature projects may be more selective**, prioritizing quality and stability over openness. As a result, there is a tension between **inclusivity (accepting more contributors)** and **maintaining high standards**, especially in highly visible or critical projects.

Overall, the findings highlight that developers must navigate trade-offs when choosing where to contribute. While active teams offer more opportunities, highly popular teams may be harder to join. Thus, joining behavior is shaped by both **developer effort and team receptivity**, with successful onboarding depending on aligning with teams that balance opportunity and accessibility.

4 Implications

Community and Website Design The findings suggest that OSS platforms and communities should actively support **both technical and social forms of contribution**, as successful onboarding depends on a combination of coding activity and interaction. Platforms can improve developer onboarding by providing **clear contribution guidelines, visible participation metrics, and structured pathways (e.g., checklists or tutorials)** that reflect common successful behaviors. Additionally, interfaces can be enhanced to **highlight meaningful interactions such as pull requests and discussion engagement**, helping both developers track their progress and teams identify promising contributors. By making contribution patterns more transparent and actionable, platforms can **lower the barrier to entry while maintaining quality**, ultimately improving the sustainability of OSS ecosystems.

Educators For educators, the results emphasize that preparing students for OSS participation requires more than teaching programming skills. Successful contributors must also develop **collaborative and communication abilities**, including engaging in discussions, reviewing others' work, and understanding community norms. Educational curricula should therefore incorporate **real-world OSS interaction practices**, encouraging students to participate in issue discussions, contribute incrementally, and build long-term engagement with projects. This approach shifts the focus from isolated coding proficiency to **holistic community involvement**, better equipping students for modern, collaborative software development environments.

5 Limitations

A primary limitation of the study lies in the **incompleteness and bias of the dataset**. Since the dataset includes only developers who had submitted at least one pull request before the initial snapshot, it excludes contributors who joined teams through other pathways. This creates a sampling bias that may not fully represent all types of OSS participants. Additionally, the study cannot precisely determine the exact timing of when a developer joined a team, instead relying on a broad time window between snapshots, which limits the granularity of behavioral analysis.

Another important limitation is the **lack of insight into developer intent**. The study identifies whether a developer joined a team but cannot determine whether non-joining was due to rejection, lack of interest, or external factors. Without understanding motivations, the analysis captures only observable behavior and may overlook nuanced differences between intentional and incidental outcomes. Furthermore, the study does not account for **off-platform interactions**, such as communication through mailing lists or external tools, which are common in OSS communities and may significantly influence joining decisions.

Finally, the study relies on simplified representations of complex social structures, such as the **core periphery model**, which may not perfectly align with real-world OSS dynamics. Differences between volunteer contributors and paid developers are not explicitly captured, and role definitions on GitHub may not fully reflect actual influence or responsibility within a project. Additionally, some metrics may be correlated, potentially introducing **collinearity** [2] **effects** that affect the interpretation of results. These factors limit the generalizability of the findings across all OSS contexts.

6 Future Work

Future work should focus on addressing the observed tensions and gaps in the current analysis by exploring **finer-grained behavioral patterns and contextual factors**. This includes examining the content and quality of contributions, understanding developer intentions, and analyzing interactions beyond GitHub. Additionally, studying how behaviors evolve over time and across different types of contributors (e.g., volunteers vs. paid developers) can provide a more nuanced understanding of onboarding dynamics. Expanding the analysis to include **content-based and social network perspectives** would further enhance insights into how developers successfully integrate into OSS teams.

Another promising direction for future work is to explore the **temporal evolution of developer behavior and team dynamics**. Instead of relying on coarse snapshots, future studies could leverage fine-grained timelines to analyze how specific sequences of actions—such as the progression from discussions to pull requests—affect joining outcomes. This would enable a deeper understanding of **causal pathways and critical milestones in the onboarding process**. Additionally, incorporating longitudinal analysis could reveal how sustained engagement, bursts of activity, or periods of inactivity influence both developer success and team receptivity, providing more actionable insights for designing better OSS ecosystems.

References

[1] Kumiyo Nakakoji et al. "Evolution Patterns of Open-Source Software Systems and Communities". In: *International Workshop on Principles of Software Evolution (IWPSE)* (Jan. 2003). DOI: 10.1145/512035.512055.

[2] *The Open Source Definition*. URL: https://opensource.org/osd.

[3] Brian Fitzgerald. "The Transformation of Open Source Software". In: *MIS Quarterly: Management Information Systems* 30 (Sept. 2006). DOI: 10.2307/25148740.

[4] Zakir Durumeric et al. "The Matter of Heartbleed". In: Nov. 2014, pp. 475–488. DOI: 10.1145/2663716.2663755.

[5] Christian Bird et al. "Open Borders? Immigration in Open Source Projects". In: June 2007, pp. 6–6. ISBN: 0-7695-2950-X. DOI: 10.1109/MSR.2007.23.

[6] Development Apache et al. "Two Case Studies of Open Source Software Development: Apache and Mozilla". In: *ACM Transactions on Software Engineering and Methodology* 11 (Sept. 2002). DOI: 10.1145/567793.567795.

[7] Dinh Trung Truong and James Bieman. "The FreeBSD project: a replication case study of open source development". In: *Software Engineering, IEEE Transactions on* 31 (July 2005), pp. 481–494. DOI: 10.1109/TSE.2005.73.

[8] Nicolas Ducheneaut. "Socialization in an Open Source Software Community: A Socio-Technical Analysis". In: *Computer Supported Cooperative Work* 14 (Aug. 2005), pp. 323–368. DOI: 10.1007/s10606-005-9000-1.

[9] Bogdan Vasilescu, Vladimir Filkov, and Alexander Serebrenik. "Perceptions of Diversity on Git Hub: A User Survey". In: May 2015, pp. 50–56. DOI: 10.1109/CHASE.2015.14.

[10] Sonali Shah. "Motivation, Governance, and the Viability of Hybrid Forms in Open Source Software Development". In: *Management Science* 52 (July 2006), pp. 1000–1014. DOI: 10.2139/ssrn.898247.

[2] Collinearity effects (often called multicollinearity) occur in statistical models when two or more independent variables are highly correlated with each other, meaning they carry similar information.

[11] Cleidson Souza, Jon Froehlich, and Paul Dourish. "Seeking the source: Software source code as a social and technical artifact". In: Jan. 2005, pp. 197–206. DOI: 10.1145/1099203.1099239.

[12] Kevin Crowston and Ivan Shamshurin. "Core-periphery communication and the success of free/libre open source software projects". In: *Journal of Internet Services and Applications* 8 (July 2017). DOI: 10.1186/s13174-017-0061-4.

[13] Mitchell Joblin et al. "Classifying Developers into Core and Peripheral: An Empirical Study on Count and Network Metrics". In: (Apr. 2016). DOI: 10.48550/arXiv.1604.00830.

[14] Georg von Krogh, Sebastian Spaeth, and Karim R Lakhani. "Community, joining, and specialization in open source software innovation: a case study". In: *Research Policy* 32.7 (2003). Open Source Software Development, pp. 1217–1241. ISSN: 0048-7333. DOI: https://doi.org/10.1016/S0048-7333(03)00050-7. URL: https://www.sciencedirect.com/science/article/pii/S0048733303000507.

[15] Vibha Singhal Sinha, Senthil Mani, and Saurabh Sinha. "Entering the circle of trust: developer initiation as committers in open-source projects". In: *Proceedings of the 8th International Working Conference on Mining Software Repositories, MSR 2011 (Co-located with ICSE), Waikiki, Honolulu, HI, USA, May 21-28, 2011, Proceedings*. Ed. by Arie van Deursen, Tao Xie, and Thomas Zimmermann. ACM, 2011, pp. 133–142. DOI: 10.1145/1985441.1985462. URL: https://doi.org/10.1145/1985441.1985462.

[16] Casey Casalnuovo et al. "Developer onboarding in GitHub: the role of prior social links and language experience". In: *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015, Bergamo, Italy, August 30 - September 4, 2015*. Ed. by Elisabetta Di Nitto, Mark Harman, and Patrick Heymans. ACM, 2015, pp. 817–828. DOI: 10.1145/2786805.2786854. URL: https://doi.org/10.1145/2786805.2786854.

[17] Jyoti Sheoran et al. "Understanding "watchers" on GitHub". In: *11th Working Conference on Mining Software Repositories, MSR 2014, Proceedings, May 31 - June 1, 2014, Hyderabad, India*. Ed. by Premkumar T. Devanbu, Sung Kim, and Martin Pinzger. ACM, 2014, pp. 336–339. DOI: 10.1145/2597073.2597114. URL: https://doi.org/10.1145/2597073.2597114.

[18] Israel Herraiz et al. "The processes of joining in global distributed software projects". In: *Proceedings of the 2006 International Workshop on Global Software Development For the Practitioner, GSD '06, Shanghai, China, May 23, 2006*. Ed. by Philippe Kruchten et al. ACM, 2006, pp. 27–33. DOI: 10.1145/1138506.1138513. URL: https://doi.org/10.1145/1138506.1138513.

[19] Laura A. Dabbish et al. "Social coding in GitHub: transparency and collaboration in an open software repository". In: *CSCW '12 Computer Supported Cooperative Work, Seattle, WA, USA, February 11-15, 2012*. Ed. by Steven E. Poltrock et al. ACM, 2012, pp. 1277–1286. DOI: 10.1145/2145204.2145396. URL: https://doi.org/10.1145/2145204.2145396.

[20] Jennifer Marlow, Laura Dabbish, and James D. Herbsleb. "Impression formation in online peer production: activity traces and personal profiles in github". In: *Computer Supported Cooperative Work, CSCW 2013, San Antonio, TX, USA, February 23-27, 2013*. Ed. by Amy S. Bruckman et al. ACM, 2013, pp. 117–128. DOI: 10.1145/2441776.2441792. URL: https://doi.org/10.1145/2441776.2441792.

[21] Raphael Pham et al. "Creating a shared understanding of testing culture on a social coding site". In: *2013 35th International Conference on Software Engineering (ICSE)*. 2013, pp. 112–121. DOI: 10.1109/ICSE.2013.6606557.